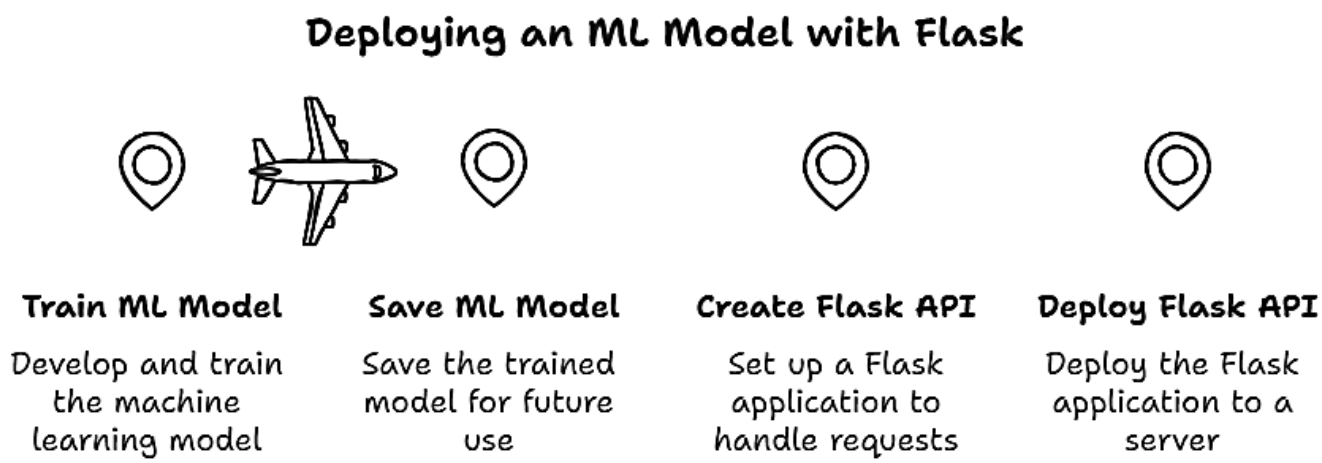
ML Model Deployment in Flask

Deploying a Machine Learning (ML) model using Flask involves several steps: training the model, saving it, creating a Flask API, and deploying it. Below is a step-by-step guide, including code snipp



Step 1: Train and Save the ML Model

Before deploying, you need an ML model. Let's train a simple model using scikit-learn **for predicting house prices.**

Install Dependencies

```
pip install numpy pandas scikit-learn flask gunicorn
```

```
pip install flask scikit-learn pandas numpy joblib
```

Step 1: Train and Save the ML Model

We will create a simple linear regression model to predict house prices based on a dataset.

Train and Save the Model (`train_model.py`):

House Price Prediction Model

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import joblib

# Sample dataset
data = {
    "area": [1500, 1800, 2400, 3000, 3500],
    "bedrooms": [3, 4, 3, 5, 4],
    "age": [10, 15, 20, 5, 8],
    "price": [300000, 400000, 500000, 600000, 650000]
}

df = pd.DataFrame(data)

# Features and target variable
X = df[["area", "bedrooms", "age"]]
y = df["price"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Save the model
joblib.dump(model, "house_price_model.pkl")

print("Model training complete and saved as 'house_price_model.pkl'.")
```

After running this script, `house_price_model.pkl` will be created, which will be used for deployment.

Step 2: Create a Flask API

Now, let's create a Flask application that will load the trained model and serve predictions.

Install Flask (if not installed):

```
pip install flask
```

Create `app.py` (Flask API):

House Price Prediction API

```python

        # Extract features
        area = data["area"]
        bedrooms = data["bedrooms"]
        age = data["age"]

        # Convert input into NumPy array and reshape
        input_features = np.array([[area, bedrooms, age]])

        # Make prediction
        prediction = model.predict(input_features)

        # Return prediction result
        return jsonify({"predicted_price": prediction[0]})

    except Exception as e:
        return jsonify({"error": str(e)})

if __name__ == "__main__":
    app.run(debug=True)

```

Step 3: Run the Flask API

Start the Flask server by running:

python app.py

Another example

ML Model Deployment in Flask

Step 1: Train and Save the ML Model

Install Dependencies

```bash
pip install scikit-learn pandas joblib
```

Create train_model.py

```python
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
import joblib

# Generate synthetic data
X, y = make_regression(n_samples=1000, n_features=5, noise=0.1, random_state=42)

# Train model
model = LinearRegression()
model.fit(X, y)

# Save model
joblib.dump(model, 'house_price_model.pkl')
print("Model saved successfully!")
```

Execute with:

```bash
python train_model.py
```

Step 2: Create Flask API

Install Flask

```bash
pip install flask
```

Create app.py

```python
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)

# Load model
model = joblib.load('house_price_model.pkl')

@app.route('/')
def home():
    return "House Price Prediction API - Send POST request to /predict"

@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Get JSON data
        data = request.json
```

```
        features = np.array(data['features']).reshape(1, -1)

        # Make prediction
        prediction = model.predict(features)
        return jsonify({'prediction': prediction.tolist()})

    except Exception as e:
        return jsonify({'error': str(e)}), 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 3: Run and Test the API

Start the Server

```bash
python app.py
```

Test with CURL

```bash
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": [2.5, 3.4, 1.8, 0.9, 4.1]}'
```

Expected Response:

```json
{"prediction":[86.432]}
```

Key Components

1. **Model Persistence**:
    - Use `joblib` for efficient serialization of scikit-learn models
    - PKL format preserves model architecture and weights
2. **Flask Endpoints**:
    - `/` : Health check endpoint
    - `/predict` : Main prediction endpoint (POST only)
3. **Data Handling**:
    - Input validation through JSON parsing
    - Numpy array reshaping for model compatibility
    - Error handling with try-except blocks

Production Deployment Considerations

1. **Use WSGI Server**:

   ```bash
   pip install waitress
   ```

   ```python
   from waitress import serve
   serve(app, host='0.0.0.0', port=5000)
   ```

2. **Model Versioning**:
   - Maintain different model versions
   - Use semantic versioning (v1.0, v2.1 etc.)
3. **Monitoring**:
   - Add logging middleware
   - Track request/response metrics
   - Implement rate limiting

Important Notes

- Always disable debug mode (`debug=False`) in production
- Use POST instead of GET for prediction endpoints (data security)
- Validate input data format and ranges
- Consider adding API key authentication for production systems
- Keep model file outside version control (add to .gitignore)

This implementation provides a basic template that can be extended with:

- Database integration for request logging
- Swagger documentation
- Docker containerization
- Load balancing for high traffic scenarios